

Interaction System Lite — Designer Manual

Version: 1.0

Unity Requirements: Unity 6 (6000.x) or newer

Author: *(your studio name)*

Category: Tools / Interaction

1. Introduction

Interaction System Lite is a lightweight, designer-friendly interaction framework for Unity. It gives teams the ability to quickly add:

- Button interactions
- Door interactions
- Pickups
- Dialogue triggers
- Custom events

...with zero coding required.

This toolkit is intentionally minimal and modular. Designers should be able to:

- Drop a prefab
- Click “Play”
- Interact immediately

No boilerplate. No scene setup headaches. No “where do I put this script?”.



2. System Overview

The Interaction System Lite consists of four core components:

2.1 Interaction Controller

Attached to your player.

This handles:

- Checking for interactables
- Showing / hiding prompts
- Sending interaction events
- Managing detection through raycast or proximity

2.2 Detectors

Two options:

Raycast Detector

Best for:

- FPS / 3rd person games
- Camera-based targeting
- Single-object focus

Proximity Detector

Best for:

- Top-down games

- ARPGs
- Interaction volumes around the player

Creates a detection sphere that triggers when objects enter range.

2.3 Interactables

Three default types included:

- **ButtonInteractable** (simple press)
- **DoorInteractable** (animated open/close)
- **PickupInteractable** (collect items)

Designers can create custom interactables using the provided base class.

2.4 Interaction Prompt UI

A simple screen-space UI element that shows:

Press E to Interact

Designers can customize:

- Text
- Font
- Colors
- Animation
- Input key



3. Getting Started (2-Minute Setup)

Open:

Tools → Interaction System Lite → Setup Wizard

Follow each step:

Step 1 — Add Interaction Controller

Select your player in the hierarchy.
Click **Add Interaction Controller**.

This injects the core system.
Required for all interactables.

Step 2 — Create Interaction Prompt UI

Click **Create Interaction Prompt UI**.

A new UI element appears under your scene Canvas:

```
Canvas
├── InteractionPromptUI
│   └── PromptText
```

Customize it like any normal UI element.

Step 3.a — Add Detectors

Pick one:

- **Add Raycast Detector**
- **Add Proximity Detector**

Raycast Detector

Adds a forward ray from your camera.

Proximity Detector

Creates a child object with a SphereCollider used as the detection volume.

Hierarchy example:

```
Player
├── ProximityDetector
│   ├── SphereCollider (Trigger)
│   └── ProximityDetector.cs
```

Step 3.b — Create Interactable Prefabs

Click the prefab creation buttons:

- Button
- Door
- Pickup

This generates ready-to-use prefabs:

[Assets/InteractionSystemLite/Prefabs/](#)

Drag them into your scene → done!

Step 4 — Import Demo Scenes

Two sample scenes demonstrate:

- Button Interaction
- Dialogue Interaction

Great for designers to reference.

4. Interactable Types (Designer Controls)

Each interactable includes a **simple inspector** with designer-friendly settings.

4.1 ButtonInteractable

Inspector options:

- Interaction Text (ex: Press E to Open)
- Events On Interact
- Highlight Color
- Play Audio on Press

Designer example:

```
ButtonInteractable
  Interaction Text: "Press E"
  OnInteract: PlayAnimation(Press)
              TriggerEvent(OpenDoor)
```

4.2 DoorInteractable

Inspector options:

- Open Angle
- Open Speed

- Auto-close?
 - Auto-close Delay
 - Locked State
-

4.3 PickupInteractable

Inspector options:

- Item Name
- Auto Destroy on Pickup
- OnPickup event

Perfect for:

- Keys
 - Loot
 - Inventory items
-



5. How Interaction Works

There are three parts:

5.1 Detection

Either raycast or spherical volume:

-  **Raycast** → selects the object in front of camera
-  **Proximity** → selects the nearest object in trigger range

5.2 Prompt Display

If an interactable becomes “active,”
Interaction Controller shows the prompt:

[E] Open Door

5.3 Interaction Event

Designer just defines what happens:

OnInteract → (Event)

Examples:

- Open animation
 - Play sound
 - Fire custom UnityEvent
 - Activate timeline
 - Load scene
 - Add item
 - Toggle lights
-



6. Creating Custom Interactables

Designers can request new interactions easily.

Developers extend:

```
public class MyCustomInteractable : BaseInteractable
{
    public override void Interact(InteractionController controller)
    {
        // Custom behavior here
    }
}
```

And now the designer can place the new interactable in the scene.



7. Designer Troubleshooting Guide

Prompt doesn't appear

- Ensure **InteractionPromptUI** exists
 - Ensure Player has **InteractionController**
 - Ensure the interactable has a collider
 - Ensure Raycast Detector has a valid camera
-

Cannot interact

- Player may be too far
 - Raycast may not hit due to layers
 - Proximity collider radius too small
-

Door animates incorrectly

- Check pivot alignment
 - Check open angle direction
 - Check animation speed
-

Pickup does nothing

- Check OnInteract event
 - Ensure auto-destroy not disabled
-

8. Designer Tips & Best Practices

- Always prefab interactables
 - Use clear naming conventions (Door_A, Key_CrystalBlue, etc.)
 - Keep interaction text short
 - Use color coding for prompts (blue = neutral, yellow = attention, red = locked)
 - Test interactions early in greybox levels
-

9. Versioning & Support

Include:

- Changelog
- Known Issues

- Supported Unity versions
-

10. Glossary (Designer-Friendly)

Interactable: Anything the player can interact with

Prompt: The UI telling the player what action is available

Controller: Script that reads input and triggers interactions

Detector: How the player finds interactables (ray or proximity)

Interaction Event: The thing that happens when you interact